

LAPORAN PRAKTIKUM
INTERNET OF THINGS (IOT) II
MODUL 4



Disusun Oleh:

Nama : Ambar Wati

Npm : 25782003

Kelas : 3A

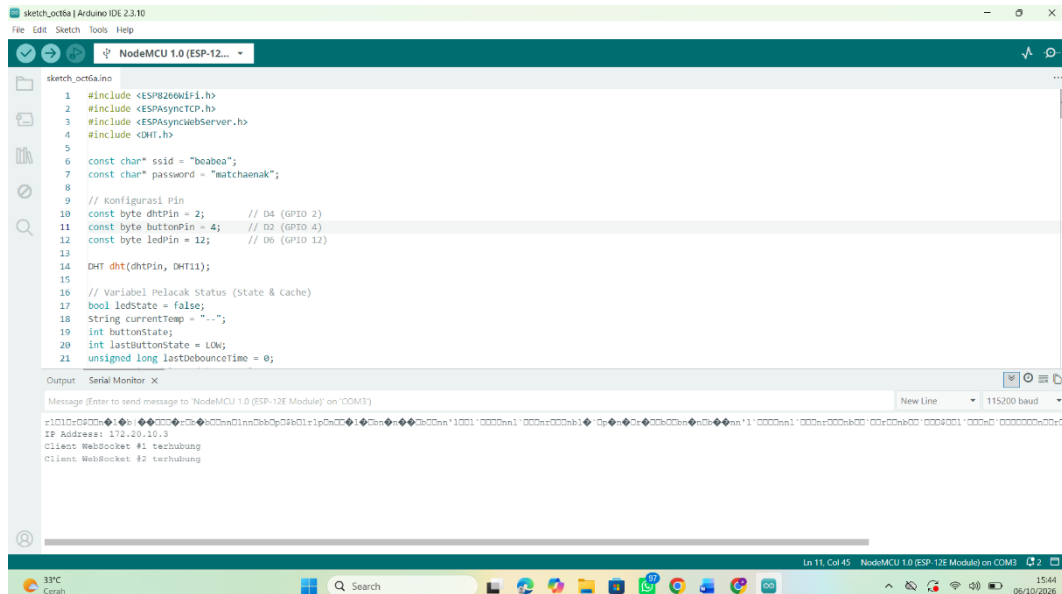
Dosen Pengampu

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

JURUSAN TEKNOLOGI INFORMASI
PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET
POLITEKNIK NEGERI LAMPUNG
2026/2027

Praktikum 4.5.1 Praktikum 1: *Real-Time Dashboard* & Sinkronisasi Tombol Fisik



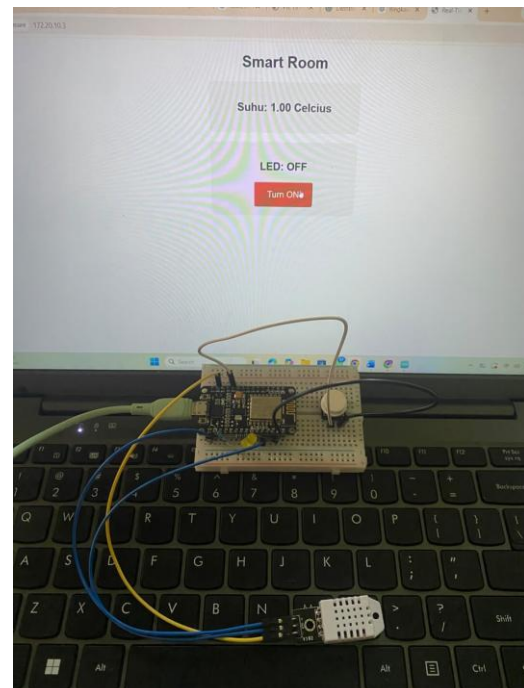
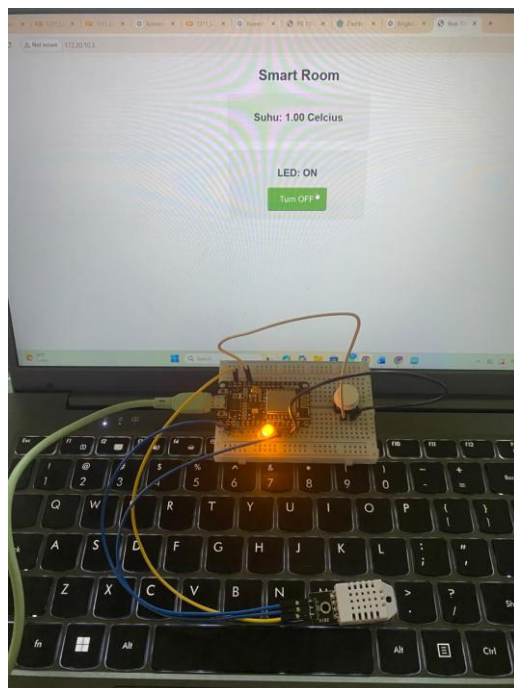
```
1 #include <ESP8266WiFi.h>
2 #include <ESPAsyncTCP.h>
3 #include <ESPAsyncWebServer.h>
4 #include <DHT.h>
5
6 const char* ssid = "beabea";
7 const char* password = "matchaenak";
8
9 // Konfigurasi Pin
10 const byte dhtPin = 2; // D4 (GPIO 2)
11 const byte buttonPin = 4; // D2 (GPIO 4)
12 const byte ledPin = 12; // D6 (GPIO 12)
13
14 DHT dht(dhtPin, DHT11);
15
16 // Variabel Pelacak Status (State & Cache)
17 bool ledState = false;
18 String currentTemp = "-.-";
19 int buttonState;
20 int lastbuttonState = LOW;
21 unsigned long lastDebounceTime = 0;
```

Output: Serial Monitor X

Message [Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3']

New Line 115200 baud

IP Address: 172.20.10.3
Client Websocket #1 terhubung
Client Websocket #2 terhubung



Penjelasan:

Pada praktikum ini dibuat sistem kontrol LED menggunakan push button dan web secara real-time. Saat tombol web ditekan, LED dan status pada browser langsung berubah. Begitu juga saat push button ditekan, status LED pada web ikut berubah tanpa perlu refresh halaman. Penggunaan WebSocket membuat perubahan status dapat dikirim langsung ke semua browser yang terhubung, sedangkan `millis()` digunakan agar pembacaan sensor DHT tidak mengganggu proses lainnya. Hasil

pengujian menunjukkan bahwa LED, push button, dan web berhasil tersinkronisasi secara real-time.

4.6

Latihan 1

Pada praktikum ini, WebSocket lebih hemat bandwidth karena koneksi antara browser dan ESP tetap terbuka, sehingga data hanya dikirim ketika ada perubahan. Sedangkan Meta Refresh HTTP akan meminta dan memuat ulang seluruh halaman secara berkala, sehingga data yang dikirim lebih banyak.

WebSocket juga tidak membuat layar berkedip karena halaman HTML tidak perlu dimuat ulang. Data yang berubah cukup diperbarui pada bagian tertentu saja. Jadi, WebSocket lebih cocok untuk sistem yang membutuhkan respon real-time dan cepat seperti kontrol LED pada praktikum ini.

Latihan 2

Pada praktikum ini, jika menggunakan `delay(3000)` di dalam `void loop()`, NodeMCU akan berhenti selama 3 detik setiap kali menjalankan perintah tersebut. Akibatnya, komunikasi WebSocket menjadi lambat karena NodeMCU tidak bisa langsung memproses perintah dari browser.

Dampaknya, tombol web dan push button terasa terlambat merespons, status LED tidak langsung berubah, dan komunikasi dengan browser bisa terganggu. Karena itu digunakan `millis()` agar pembacaan sensor tetap berjalan tanpa menghentikan proses WebSocket.

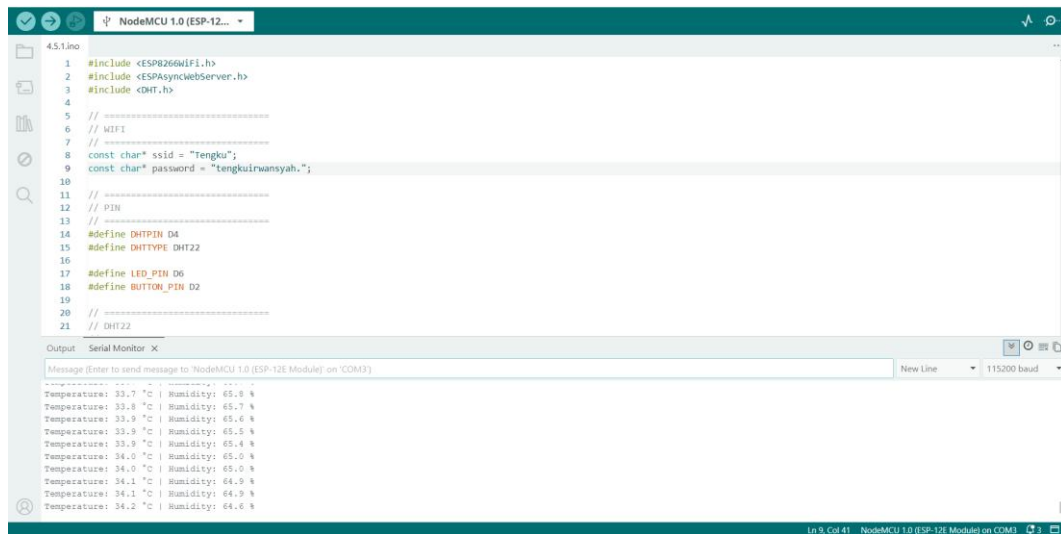
Latihan 3

Pada praktikum ini, JavaScript digunakan untuk mengubah tampilan pada web sesuai dengan status hardware. ID elemen HTML yang menjadi target perubahan teks adalah:

- `ledState` → menampilkan status LED, seperti ON/OFF.
- `ledStatus` → menampilkan keterangan kondisi LED.
- `temperature` → menampilkan nilai suhu dari sensor DHT.
- `humidity` → menampilkan nilai kelembapan dari sensor DHT.

Jadi, script akan memperbarui isi `<span>` tersebut secara otomatis ketika menerima data terbaru dari NodeMCU.

Latihan 4

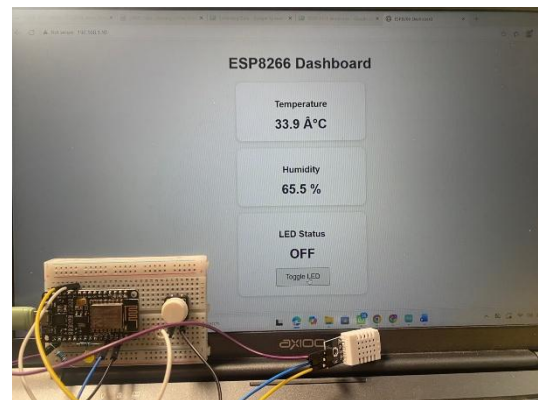
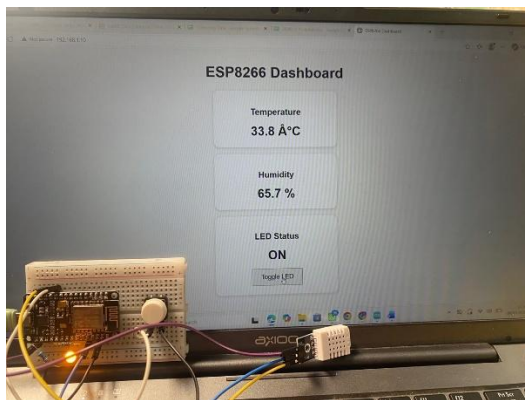


```
4.5.1.ino
1 #include <ESP8266WiFi.h>
2 #include <ESPAsyncWebServer.h>
3 #include <DHT.h>
4
5 // =====
6 // WIFI
7 // =====
8 const char* ssid = "Tengku";
9 const char* password = "tengkuiruanyah.";
10
11 // =====
12 // PIN
13 // =====
14 #define DHTPIN D4
15 #define DHTTYPE DHT22
16
17 #define LED_PIN D6
18 #define BUTTON_PIN D2
19
20 // =====
21 // DHT22
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

Temperature: 33.7 °C | Humidity: 65.8 %
Temperature: 33.8 °C | Humidity: 65.7 %
Temperature: 33.9 °C | Humidity: 65.6 %
Temperature: 33.9 °C | Humidity: 65.5 %
Temperature: 33.9 °C | Humidity: 65.4 %
Temperature: 34.0 °C | Humidity: 65.0 %
Temperature: 34.0 °C | Humidity: 65.0 %
Temperature: 34.1 °C | Humidity: 64.9 %
Temperature: 34.1 °C | Humidity: 64.9 %
Temperature: 34.2 °C | Humidity: 64.6 %



Penjelasan

Pada praktikum ini, program dimodifikasi agar tidak hanya menampilkan suhu, tetapi juga kelembapan dari sensor DHT secara real-time. Pada bagian Back-End ditambahkan variabel `currentHum` untuk menyimpan nilai kelembapan. Nilai kelembapan dibaca menggunakan `dht.readHumidity()` dengan timer `millis()` agar proses pembacaan sensor tidak mengganggu komunikasi WebSocket.

Data kelembapan kemudian ditambahkan ke dalam JSON yang dikirim oleh NodeMCU ke browser. Pada bagian Front-End, ditambahkan kartu Kelembapan dengan `id="humValue"`. JavaScript akan mengambil data `hum` dari JSON dan menampilkan nilainya pada halaman web.

Dengan modifikasi ini, suhu dan kelembapan dapat ditampilkan secara real-time pada web tanpa perlu melakukan refresh halaman.